

AI Agent 沙箱网络安全

从流量隔离到智能治理

演讲人：王炳燊 李博康

阿里云

AiCon

全球人工智能开发与应用大会

目录

01

第一章

威胁模型与策略设计

面向不可信代码的沙箱安全挑战，从威胁模型出发，设计分层的流量管控策略。

02

第二章

工程实现与性能验证

透明的流量接管与凭据注入、经大规模验证的性能，以及开放共建的开源生态。

AI Agent 爆发式增长

短短三年，AI 从只会对话的助手，演进为能执行代码、调用工具的自治 Agent。



前所未有的安全挑战

攻击路径示例：Agent 访问 MetaServer 获取云账号

- 1 用户提交 Prompt → LLM 生成代码
- 2 代码执行 `curl 100.100.100.200` (ECS MetaServer)
- 3 获取 STS Token (RAM 临时凭证)
- 4 用 Token 调用 OpenAPI → 拿到 OSS/RDS 等资源权限
- 5 **横向移动 → 影响同账号下所有资源**

核心问题：如何为AI Agent沙箱构建不可绕过的网络边界？

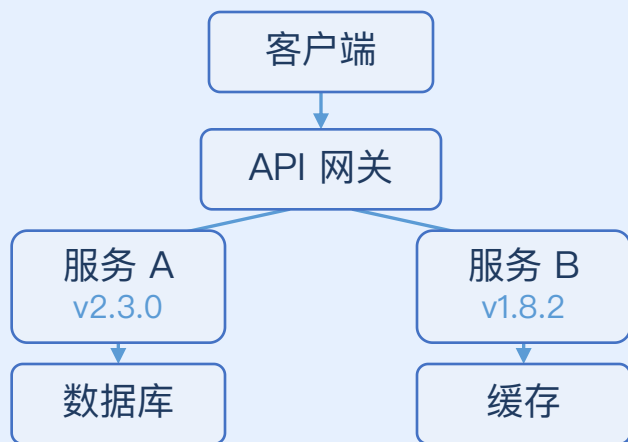
01 工作负载的变化

代码沙箱正在从隔离应用转向管控不可信 AI 生成代码

Agent沙箱从“运行应用”变成“运行不可信代码”

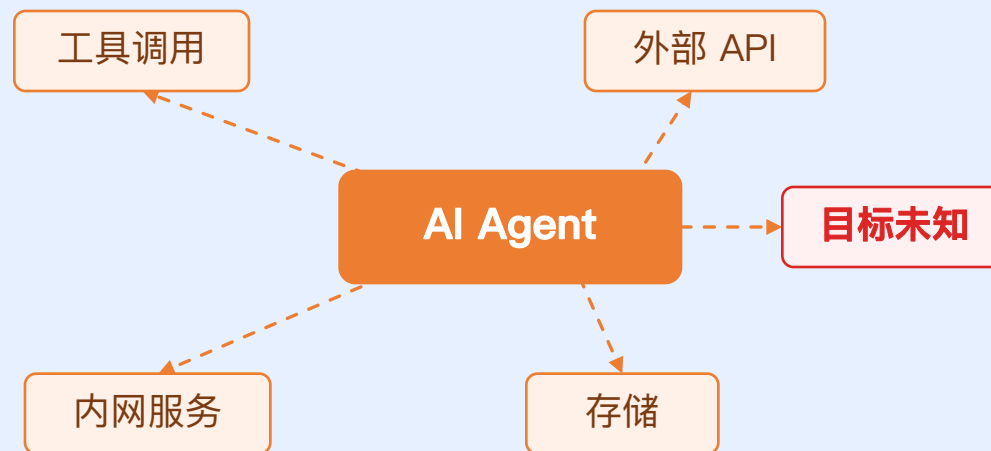
传统工作负载

- 应用代码相对稳定
- 访问目标可预期
- 服务间调用可建模
- 变更节奏按发布推进



AI Agent沙箱

- 调用关系动态
- 访问目标未知



Agent workload把目标、身份和风险都推迟到运行时

01 不可信代码

执行 LLM 生成代码或用户上传脚本，不能假设其遵守平台 SDK、代理地址或网关约束。



02 动态工具调用

访问目标取决于任务、提示词和工具链，部署前很难枚举完整目标集合。



03 多租横向风险

同一集群中可能同时运行多租户沙箱，存在横向探测和未授权访问风险。



04 短生命周期

沙箱启动快、销毁快，策略必须在首包前生效，不能等慢速 sidecar 或人工配置收敛。



■ 传统NetworkPolicy的局限性

NetworkPolicy 能力	只能回答	Agent 场景差距
Pod / Namespace selector	哪些 Pod 被策略选中	只理解 Kubernetes 标签
Ingress / Egress	入站或出站是否允许	方向内仍是 L3/L4 语义
IPBlock / Port	某个网段、端口是否可达	不理解域名和请求语义
白名单模型	未显式允许则拒绝	策略数量随沙箱规模增长

NetworkPolicy 是必要的 L3/L4 基线，但无法覆盖 Agent 场景所需的 FQDN 白名单、身份语义和 L7 请求级管控，需要叠加更高层策略

Agent场景下需要身份、L7相关语义

01

域名白名单

工具以域名声明目标，
IP 随时可变，需基于域
名管控

02

HTTP Path / Header

按请求路径和 Header
区分 API 调用权限

03

Token / AK 管控

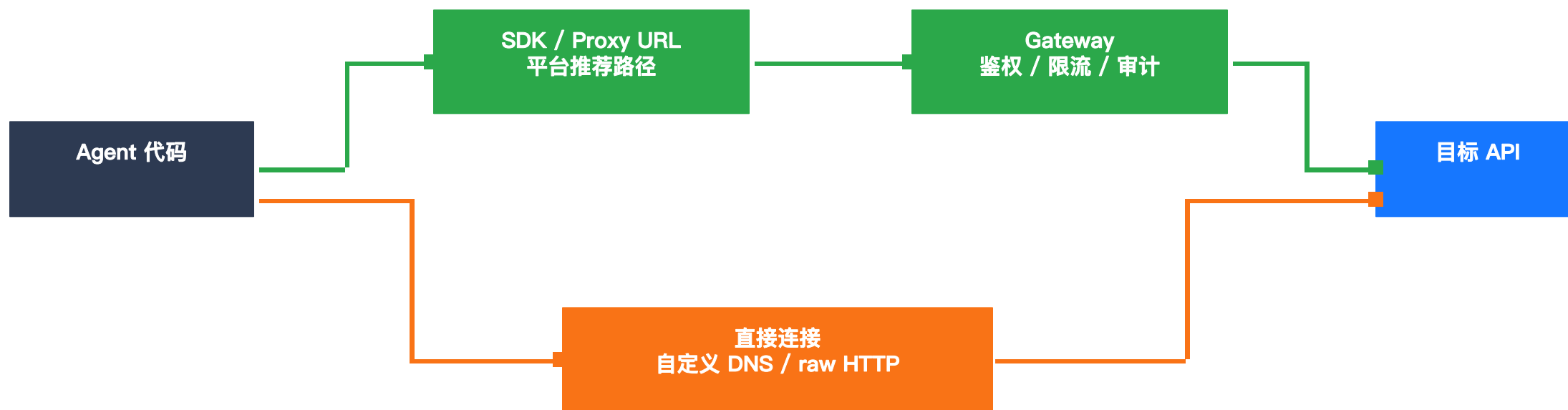
凭证在网关动态注入，
不暴露给不可信代码

04

内容审计

LLM 请求与响应需合规
过滤与记录

■ 现有API/LLM/MCP网关依赖主动接入，恶意代码可以绕过



网关仍然有价值，但其安全前提是「**client 自愿走网关**」——不可信代码不受这一约束。

■ 强安全边界必须非协作、透明、首包前生效、默认受控

01 非协作式

不要求业务代码使用 SDK、代理地址或网关域名；绕过推荐路径仍会被平台控制。

02 透明拦截

在网络路径上强制接管 HTTP/HTTPS 流量，而不是等待 client 配置正确。

03 首包前生效

沙箱启动时策略必须先于业务连接就绪，避免冷启动窗口暴露。

04 默认受控

平台基线先执行，租户只能在受控边界内叠加工具和域名白名单。

■ 分层策略构建安全边界

L4 平台基线

集群级 · 平台强制

封禁元数据 API (100.100.100.200)、高危端口、内网网段。优先级最高，租户策略无法覆盖。

→ 满足"默认受控"
平台基线先于一切租户配置

L4 租户策略

命名空间级 · 按需叠加

按需FQDN / Service / CIDR / Workload 白名单，端口协议过滤。DNS TTL 驱动自动刷新。

→ 满足"非协作 + 透明拦截"
nftables/BPF 内核态执行

L7 请求级授权

工作负载级 · 身份 + 内容

HTTP Method/Path/Header ACL、凭据注入 (Token 替换)、TLS 终止、审计 Webhook。

→ 满足"身份 + L7 语义"
管控到 API 粒度

02 Agent 流量全栈管控

TrafficPolicy + SecurityProfile



三层策略协同：Sandbox 流量全栈管控

由外到内：平台不可绕过的基线（L4）→ 租户按需声明（L4）→ 工作负载身份+请求级授权（L4+L7）

GlobalTrafficPolicy 集群级 · 平台强制 · 租户不可覆盖

集群全局

L4 封禁全局高危 IP / 元数据 API / 端口（不可绕过）

TrafficPolicy 租户级 · 按需声明 · 叠加在全局基线之上

命名空间

L4 CIDR / FQDN / Service 白名单、端口协议过滤

SecurityProfile 工作负载级 · 身份认证 + 请求级授权

工作负载

L4 mTLS Peer 认证（ServiceAccount） / TLS 强制 / Pod 隔离

L7 JWT / OIDC 授权 / HTTP Method & Path ACL / gRPC 方法拦截

Sandbox Pod LLM 生成代码 / 工具调用

GlobalTrafficPolicy 在 Admission 阶段注入，策略优先级最高，不可被租户 TrafficPolicy 或 SecurityProfile 覆盖。三者由外到内叠加生效，均在首包前完成策略下发。

TrafficPolicy 和 SecurityProfile 各管什么

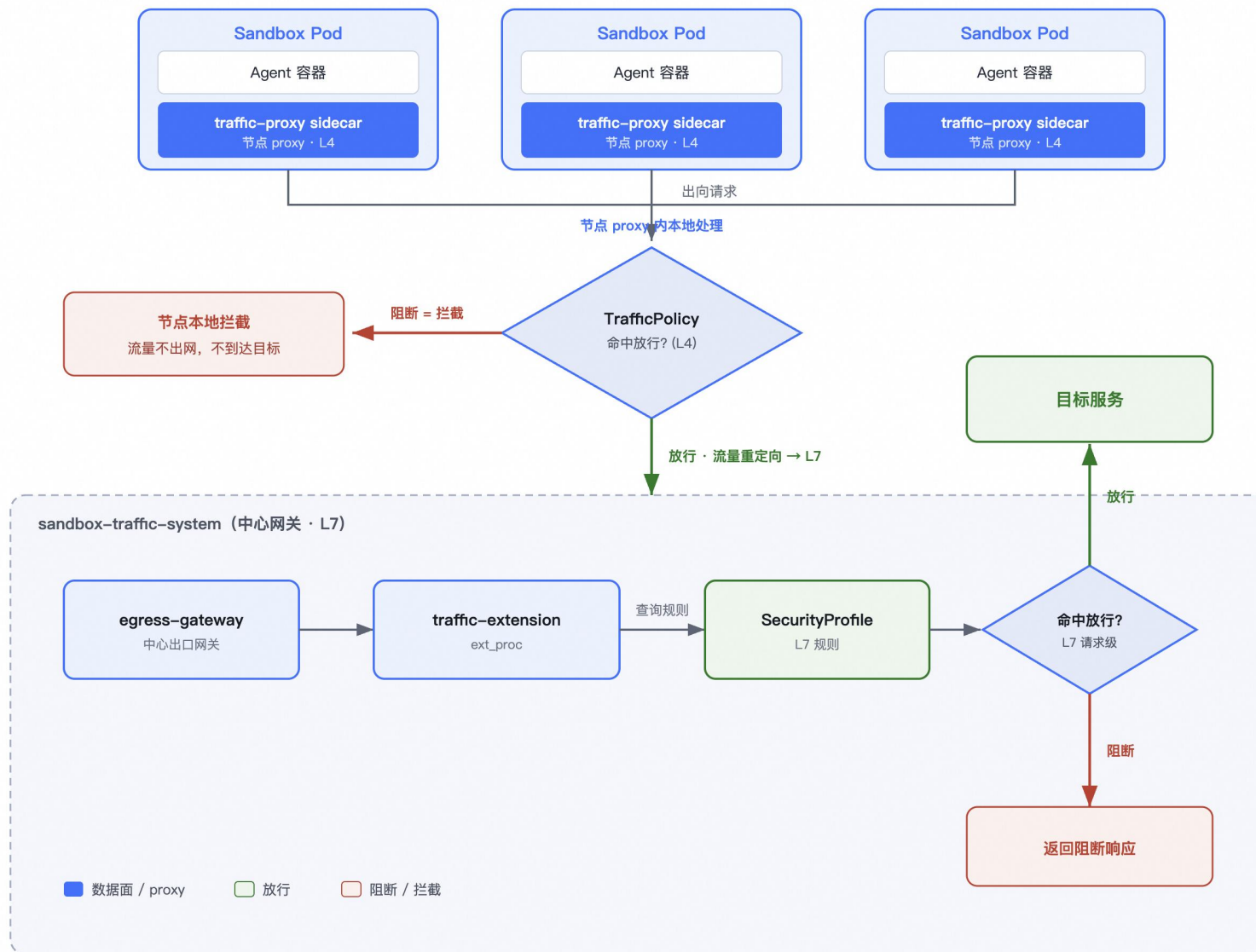
TrafficPolicy 流量管控

- 目标 IP / CIDR 白名单
- 端口、协议过滤 (TCP、UDP)
- Service / FQDN 解析放行
- 默认拒绝 (Deny-all) 兜底

SecurityProfile 安全访问控制

- 身份认证mTLS Peer 认证、TLS Terminate
- HTTP Method/Path/Header ACL策略
- 凭据管理自定义Token 替换, OpenAPI STS Token 替换
- 审计与webhook通知

部署架构



■ 技术选型与演进：为什么是中心化的网关

网关从节点走到中心

一开始网关放在每个节点上，就近转发、链路短。后来要做凭据替换，真实的 AK/Token 不能下放到节点和沙箱，否则暴露面太大，节点上也不好统一管——于是把网关收到了中心。

可观测、可运维同一个方向

网关分散在各节点，日志、审计、限流、配额都难统一，排障成本高；集中之后这些才好做，策略一处维护，灰度和回滚也可控。

从自研 Go 网关换成 Envoy

网关最早是 Go 写的用户态七层程序，自己维护协议和扩展太重；后来换成成熟的 Envoy，用它的 ext_proc 和插件机制，把精力放回策略本身。

TrafficPolicy Spec 核心概念

01 两种作用域

TrafficPolicy 命名空间级，写租户配置
GlobalTrafficPolicy 集群级，写平台基线

02 双向规则

Ingress/Egress 规则独立配置
支持Allow/Deny 两种配置语意

03 四类 Peer

from/to 支持 CIDR / FQDN / Service /
Workload，支持端口、协议限制，可在同
一条 rule 里混用

04 优先级合并

数值越小越优先
GlobalTrafficPolicy 用更高优先级压住租
户规则，租户白名单覆盖不了平台 deny

■ 示例：GlobalTrafficPolicy / TrafficPolicy

GlobalTrafficPolicy (平台基线)

```
apiVersion: agents.kruise.io/v1alpha1
kind: GlobalTrafficPolicy
spec:
  priority: 100
  egress:
    - action: Deny
      to:
        - ipBlock: 100.100.100.200
          # 拒绝元数据服务
```

TrafficPolicy (租户白名单)

```
apiVersion: agents.kruise.io/v1alpha1
kind: TrafficPolicy
spec:
  priority: 1000
  egress:
    - action: Allow
      to:
        - service: coredns
        - workload: gateway
          namespace: sandbox-system
          selector: {...}
```

优先级 100 比 1000 高，平台的 Deny 压住租户的 Allow。

SecurityProfile 管控能力全景

基础能力

透明拦截

TLS Termination

强制接管 TLS 握手，HTTPS 流量对规则引擎明文可见；应用层无感知、无需改代码、无法绕过

多维匹配

RuleMatch

域名通配符、路径（前缀/精确/正则）、方法、端口、请求头、查询参数

规则链

Default Continue

按声明顺序逐条评估；匹配即执行动作并继续，遇到终端动作短路中断

数据增强

身份注入

IdentityInjection

出口请求自动注入沙箱身份 Header（名称、空间、租户 ID），上游可追溯来源

凭据转换

TokenTransformation

Agent 使用占位符 Token，数据面从安全 Provider 获取真实密钥替换，进程不碰凭据

流量镜像

Mirroring

匹配的出口请求异步镜像到安全运营中心，不影响主链路延迟，支持离线审计

访问控制

安全审计

SecurityCheck

同步调外部检测服务，拦截 Prompt 注入 / 策略违规，支持降级策略

请求限流

RateLimit

按域名/路径粒度令牌桶限速，防止单 Agent 过度调用，成本控制与公平共享

拦截 / 放行 / 转发

Block · Bypass · Forwarding

Block 返回错误、Bypass 直接放行、Forwarding 重定向至内部网关

SecurityProfile Spec 结构与评估规则

01 作用对象

Selector 选中 Pod，对其 egress HTTP/HTTPS 流量生效

02 结构组成

有序 Rules 链；一条 Rule = Match + Actions，每种 action 至多配置一次。

03 匹配规则

Domains 必填，支持 "*" 和 "*.example.com" 通配；可继续叠加 Paths / Methods / Headers。多个 Match 之间是 OR，单个 Match 内的字段之间是 AND

04 评估顺序

按 Rules 顺序逐条匹配执行，遇到第一个 terminal action 才短路

■ 示例：SecurityProfile ACL（L7 访问控制）

用 block / bypass，按域名、路径、方法、请求头、端口等维度做 L7 出口访问控制；命中 block 直接返回设定的状态码与响应体。

block 阻断 + bypass 白名单兜底

```
apiVersion: agents.kruise.io/v1alpha1
kind: SecurityProfile
metadata: {name: agent-profile, ns: default}
spec:
  selector:
    matchLabels: {agent: sample}
  rules:
    - name: deny-admin      # 阻断 /admin 前缀
      match:
        - domains: ["*"]
          paths: [{type: Prefix, value: /admin}]
          methods: [GET, POST, PUT, DELETE]
      actions:
        block: {statusCode: 403, body: '{"error": "..."}'}
    - name: allow-aliyuncs  # 白名单：放行
      match: [{domains: "*.aliyuncs.com"}]
      actions: {bypass: true}
    - name: deny-all       # 兜底拒绝(置于最后)
      match: [{domains: ["*"]}
      actions: {block: {statusCode: 403}}
```

匹配维度 · match

domains (* / *.example.com) 、 paths (Prefix / Exact / Regex) 、 methods、 headers、 queryParams、 ports

动作 · actions

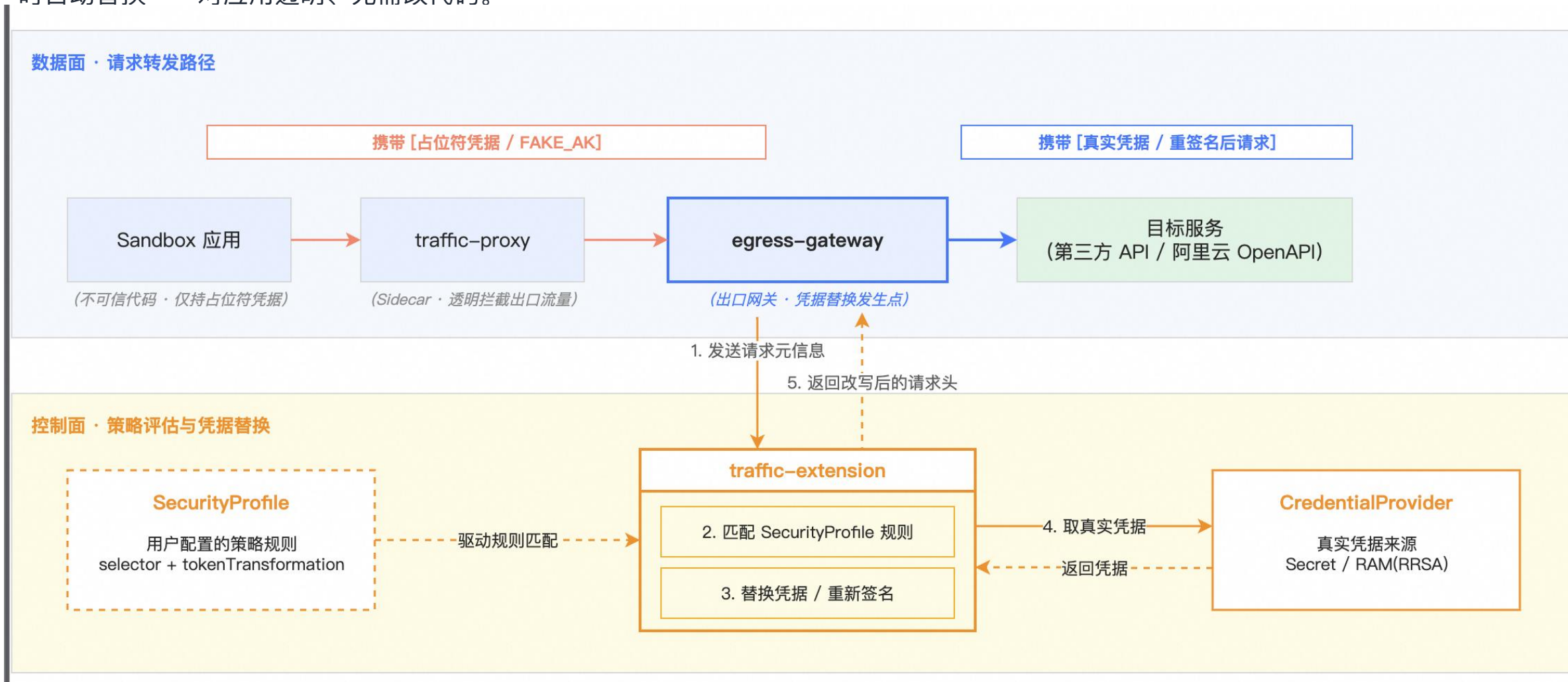
block 返回码+响应体（终止）、bypass 放行并短路（终止）、tokenTransformation 凭据注入（非终止）、audit Webhook（非终止）

匹配语义

单个 match 内字段为 AND；多个 match 之间为 OR；规则链按序评估，遇终止动作短路；默认放行（空规则=全放行）

■ 示例：凭据替换管理

Sandbox 代码不可信，直接放真实 API Key / 云 AK 有泄露风险。凭据注入让应用只拿占位符，真凭据放在平台侧，由出口网关在转发时自动替换——对应用透明、无需改代码。



■ 示例：SecurityProfile (API Key 注入)

SecurityProfile manifest

```
apiVersion: agents.kruise.io/v1alpha1
kind: SecurityProfile
metadata:
  name: inject-llm-key
spec:
  selector:
    matchLabels:
      security.agents.kruise.io/agent-name: my-agent
  rules:
    - name: inject-openai-key
      match:
        - domains:
            - "httpbin.org" # 实际替换为目标域名
      actions:
        tokenTransformation:
          type: ApiKey
```

credentialRef + apiKey

```
credentialRef:
  kind: CredentialProvider
  name: llm-api-key
apiKey:
  when:
    header: Authorization
    pattern: "^Bearer .+"
  targetHeader: Authorization
  valueTemplate: "Bearer {{ .Token }}"
```

效果：应用发 Bearer fake-token →
出口改写为真实 Bearer sk-****
真实 key 存 CredentialProvider(读 Secret)

■ 规模与性能

10万

单集群10万+ sandbox

50万条

1000 沙箱×1000 SecurityProfile×500 规则

20万条

TrafficPolicy 规则上限

25,000条

单TrafficPolicy 内规则上限

35,000 RPS

单副本（1 网关 + 1 ext, 20 SP）吞吐，
P99 < 5ms（c9i.8xlarge）

0.2 ms

ext_proc 优化：5000 RPS 下调用延迟，
P99 4.9ms

<2 s

2万规则规模下，同步生效延迟

10m 40MB

1万规则下Sidecar 开销低

0

网关、插件滚动重启、优雅下线流量无损

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



■ 社区进展与共建

四个 PR：从能用到社区可用

- **#433 API 提案**

定义 TrafficPolicy、SecurityProfile 两类 CRD

- **#445 Traffic Proxy**

升级 traffic-proxy，完善通用注入与探针

- **#448 SecurityProfile**

CRD+controller+ext_proc，Block/Bypass 可用

- **#483 SecurityProfile CRD**

CRD 定义更新

现在能用的，和还在路上的

- **已可用**

统一 API、透明注入、L7 过滤（Block/Bypass、通配、路径匹配）

- **还在做**

TrafficPolicy 独立 CRD 与 L4 数据面；IdentityInjection 跟进

- **欢迎共建**

openkruise/agents

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会